Ejercicio Aplicación

Creación Aplicación Pedido

Actualmente tenemos Pago, Usuario, Producto, pero en este proyecto de prueba que estamos desarrollando nos hace falta también los pedidos. Un pedido representa una compra realizada por un usuario y contiene uno o varios productos. Por tanto, en este ejercicio vais a crear la app `pedido` y el primer modelado real de relaciones.

Cuando definís un modelo en Django, cada "columna" se define con un Field. Un CharField representa texto corto (por ejemplo, un estado o un código). Un TextField es texto largo (observaciones). Un IntegerField/PositiveIntegerField son números enteros (por ejemplo, cantidad). Un DecimalField es el tipo correcto para dinero (precio e importes). Un BooleanField es verdadero/falso (activo, pagado, etc.). Un DateTimeField guarda fecha y hora (fecha de creación). Y para relaciones se usan campos especiales: ForeignKey (uno a muchos), OneToOneField (uno a uno) y ManyToManyField (muchos a muchos). En este ejercicio vas a necesitar ForeignKey para representar que un pedido pertenece a un usuario y que un pedido tiene varios productos mediante una entidad intermedia.

Relaciones que debéis implementar: un pedido debe poder contener varios productos. En el modelo relacional esto no se guarda con una lista dentro de una columna. Se resuelve creando una tabla intermedia (línea de pedido) que conecta un pedido con un producto y además guarda información extra como la cantidad. Por tanto, el diseño correcto es: `Pedido` tiene muchas `LineaPedido`, y cada `LineaPedido` apunta a un `Producto`. Esto se traduce en Django con dos `ForeignKey`: uno desde `LineaPedido` a `Pedido` y otro desde `LineaPedido` a `Producto`.

Ejercicio 1: (4p)

Debéis crear una app nueva llamada `pedido`. Después debéis crear los modelos dentro de la app `pedido`. Como mínimo debéis crear estos modelos: `Pedido` y `LineaPedido` (o `PedidoItem`). El modelo `Pedido` debe tener al menos un identificador (Django ya crea uno automáticamente), una fecha de creación (timestamp), un estado (texto con opciones cerradas como PENDIENTE/PAGADO/CANCELADO) y la relación con el usuario (un pedido pertenece a un usuario). El modelo `LineaPedido` debe incluir la relación con el pedido, la relación con el producto y una cantidad (entero positivo). Opcionalmente podéis añadir `precio_unitario` como `DecimalField` para guardar el precio en el momento de compra (esto es lo que se hace en empresa), pero si lo añadís, debéis justificarlo y usar el tipo correcto.

Un ejemplo a este ejercicio sería este:

```
# biblioteca/models.py  (ejemplo genérico, NO para copiar tal cual al pedido)
from django.db import models

class Socio(models.Model):
   nombre = models.CharField(max_length=120)
   def __str__(self):
       return self.nombre

class Libro(models.Model):
   titulo = models.CharField(max_length=200)
   def __str__(self):
       return self.titulo

class Prestamo(models.Model):
   ESTADOS = [
       ("ABIERTO", "Abierto"),
       ("CERRADO", "Cerrado"),
   ]
  socio =models.ForeignKey(Socio,on_delete=models.PROTECT,related_name="prestamos")
   estado = models.CharField(max_length=10, choices=ESTADOS, default="ABIERTO")
   creado_en = models.DateTimeField(auto_now_add=True)

   def __str__(self):
       return f"Préstamo #{self.id} - {self.socio.nombre}"

class PrestamoItem(models.Model):
           prestamo    =    models.ForeignKey(Prestamo,    on_delete=models.CASCADE,
related_name="items")
              libro     =     models.ForeignKey(Libro,     on_delete=models.PROTECT,
related_name="prestamos_item")
   cantidad = models.PositiveIntegerField(default=1)

   def __str__(self):
                     return   f"{self.cantidad}   x   {self.libro.titulo}   (Préstamo
#{self.prestamo.id})"
```

Ejercicio 2 (2p)

Debéis escoger los tipos de campo correctos. El estado no puede ser un número, debe ser texto (`CharField`) con opciones definidas. La cantidad no puede ser un texto, debe ser un número entero positivo (`PositiveIntegerField`). Los precios o importes nunca se guardan en `float`, si lo implementáis, se hace con `DecimalField`. La fecha de creación

se guarda con `DateTimeField(auto_now_add=True)`. La relación entre tablas se hace con `ForeignKey`, y debéis decidir el `on_delete` según el criterio de integridad: por ejemplo, si se borra un pedido, lo normal es que se borren sus líneas (`CASCADE`); si se intenta borrar un producto que ya está en pedidos, en un sistema real se protege (`PROTECT`) o se desactiva el producto, pero no se borra el histórico.

Una vez creados los modelos debéis generar y aplicar migraciones con `makemigrations` y `migrate`. Debéis poder demostrar que se han creado ejecutando `python manage.py showmigrations` y viendo la app `pedido` como aplicada.

Ejercicio 3 (2p)

Debéis crear una página web de "listado de pedidos" con plantilla y CSS que queráis de la app `pedido`. La estructura de estáticos debe ser la estándar por app:

`pedido/static/pedido/style.css` y `pedido/templates/pedido/listado.html`

Debéis cargar el CSS con `{% load static %}` y enlazarlo con `<link rel="stylesheet" href="{% static 'pedido/style.css' %}">`. En el CSS debéis aplicar un estilo visible y cuidado (fondo, tipografía, separadores, tarjetas o tabla). No se permite meter el CSS dentro del `<style>` del HTML, debe ser externo.

Ejercicio 4 (1,5p)

Crear una vista en `pedido/views.py` que consulte pedidos desde ORM y los muestre en la plantilla. Debéis crear `pedido/urls.py` y conectarlo en el `urls.py` principal del proyecto con `include()`. La ruta mínima debe ser `/pedidos/` y debe mostrar los pedidos existentes. En la plantilla, por cada pedido, debéis mostrar su estado.

Entrega (0,5p)

La entrega debe ser un archivo ZIP con el proyecto completo de Django, manteniendo la estructura original y sin carpetas innecesarias. Dentro del ZIP debe estar el `manage.py` en la raíz del proyecto, todas las apps (`usuario`, `producto`, `pago`, `pedido`), las plantillas (`templates/`), los estáticos (`static/` de cada app) y el fichero `requirements.txt` si lo estáis usando. No deben incluir el entorno virtual (carpeta `venv/`) ni carpetas de caché como `__pycache__/`, ni `.idea/` o `.vscode/`, ni ficheros temporales. Debéis añadir un Json con los datos que debo ejecutar para poder probar la migración de la BD.